Software Requirements Specification for LingoLyrics: A Song-Based Language-Learning System

Team #21, LingoLyrics
Rhea Gokhale
Aansh Kotian
Omar Bakr
Raymond Ma

October 18, 2026

Contents

1 Purpose of the Project **vi**
- 1.1 User Business . . . vi
- 1.2 Goals of the Project . . . vi

2 Stakeholders **vii**
- 2.1 Client . . . vii
- 2.2 Customer . . . vii
- 2.3 Other Stakeholders . . . vii
- 2.4 Hands-On Users of the Project . . . vii
- 2.5 Personas . . . vii
- 2.6 Priorities Assigned to Users . . . viii
- 2.7 User Participation . . . viii
- 2.8 Maintenance Users and Service Technicians . . . viii

3 Mandated Constraints **viii**
- 3.1 Solution Constraints . . . viii
- 3.2 Implementation Environment of the Current System . . . ix
- 3.3 Partner or Collaborative Applications . . . ix
- 3.4 Off-the-Shelf Software . . . ix
- 3.5 Anticipated Workplace Environment . . . ix
- 3.6 Schedule Constraints . . . ix
- 3.7 Budget Constraints . . . x
- 3.8 Enterprise Constraints . . . x

4 Naming Conventions and Terminology **x**
- 4.1 Glossary of All Terms, Including Acronyms, Used by Stakeholders involved in the Project . . . x

5 Relevant Facts And Assumptions **xi**
- 5.1 Relevant Facts . . . xi
- 5.2 Business Rules . . . xi
- 5.3 Assumptions . . . xi

6 The Scope of the Work **xii**
- 6.1 The Current Situation . . . xii
- 6.2 The Context of the Work . . . xii
- 6.3 Work Partitioning . . . xii

6.4 Specifying a Business Use Case (BUC) xii

7 Business Data Model and Data Dictionary **xiii**
7.1 Business Data Model xiii
7.2 Data Dictionary xiii

8 The Scope of the Product **xiii**
8.1 Product Boundary xiii
8.2 Product Use Case Table xiv
8.3 Individual Product Use Cases (PUC's) xiv

9 Functional Requirements **xiv**
9.1 Functional Requirements xiv

10 Look and Feel Requirements **xv**
10.1 Appearance Requirements xv
10.2 Style Requirements xv

11 Usability and Humanity Requirements **xv**
11.1 Ease of Use Requirements xv
11.2 Personalization and Internationalization Requirements xv
11.3 Learning Requirements xvi
11.4 Understandability and Politeness Requirements xvi
11.5 Accessibility Requirements xvi

12 Performance Requirements **xvi**
12.1 Speed and Latency Requirements xvi
12.2 Safety-Critical Requirements xvi
12.3 Precision or Accuracy Requirements xvi
12.4 Robustness or Fault-Tolerance Requirements xvi
12.5 Capacity Requirements xvi
12.6 Scalability or Extensibility Requirements xvi
12.7 Longevity Requirements xvii

13 Operational and Environmental Requirements **xvii**
13.1 Expected Physical Environment xvii
13.2 Wider Environment Requirements xvii
13.3 Requirements for Interfacing with Adjacent Systems xvii
13.4 Productization Requirements xvii

13.5 Release Requirements xvii

14 Maintainability and Support Requirements **xvii**
14.1 Maintenance Requirements xvii
14.2 Supportability Requirements xvii
14.3 Adaptability Requirements xviii

15 Security Requirements **xviii**
15.1 Access Requirements xviii
15.2 Integrity Requirements xviii
15.3 Privacy Requirements xviii
15.4 Audit Requirements xviii
15.5 Immunity Requirements xviii

16 Cultural Requirements **xviii**
16.1 Cultural Requirements xviii

17 Compliance Requirements **xviii**
17.1 Legal Requirements xviii
17.2 Standards Compliance Requirements xix

18 Open Issues **xix**

19 Off-the-Shelf Solutions **xix**
19.1 Ready-Made Products xix
19.2 Reusable Components xix
19.3 Products That Can Be Copied xix

20 New Problems **xix**
20.1 Effects on the Current Environment xix
20.2 Effects on the Installed Systems xix
20.3 Potential User Problems xix
20.4 Limitations in the Anticipated Implementation Environment That May Inhibit the New Product xx
20.5 Follow-Up Problems xx

21 Tasks **xx**
21.1 Project Planning xx
21.2 Planning of the Development Phases xx

22 Migration to the New Product **xxi**
22.1 Requirements for Migration to the New Product xxi
22.2 Data That Has to be Modified or Translated for the New System xxi

23 Costs **xxi**

24 User Documentation and Training **xxi**
24.1 User Documentation Requirements xxi
24.2 Training Requirements xxii

25 Waiting Room **xxii**

26 Ideas for Solution **xxii**

27 Appendix — Generative AI Use Disclosure **xxiii**
27.1 AI Tools Used xxiii
27.2 How and Where Used xxiii
27.3 Verification of AI-Generated Content xxiii

28 Appendix — Reflection **xxiv**

Revision History

Date	Version	Notes
October 5th, 2026	0.1	Initial draft alignment with project planning and requirements sources.
October 19, 2026	1.0	Complete Volere SRS initial submission.

1 Purpose of the Project

1.1 User Business

Self-directed language learners at early-intermediate to intermediate levels frequently utilize music and songs to encounter authentic vocabulary, regional idioms, and rich cultural contexts. However, standard media players and lyrics websites fail to support structured, in-context learning. When learners encounter unfamiliar vocabulary, they are forced to pause audio, switch to external dictionary tools, and manually transfer vocabulary into flashcard applications. This constant context-switching interrupts listening flow and separates words from their surrounding lyrical nuances.

LingoLyrics solves this disconnect by combining interactive, real-time synchronized lyrics, line-by-line translations, transliterations for non-Latin scripts, contextual annotations, and direct vocabulary exports into a unified workflow.

1.2 Goals of the Project

The primary goals of LingoLyrics are as follows:

- **Synchronized Lesson Playback:** Ensure real-time line-by-line lyric highlighting within 0.5 seconds of mapped timestamps across target test tracks.
- **In-Context Vocabulary Support:** Render contextual annotations, translations, and transliterations immediately upon word or line selection.
- **Creator-to-Learner Interoperability:** Maintain a shared, schema-validated song-map file format that flows seamlessly from the creator editor to the mobile learner application without manual modification.
- **Practical Lesson Creation:** Enable song-map creators to author complete, validated song maps for short audio tracks within 45 minutes using a visual editor.
- **Language-Aware Support:** Accurately segment and render lyrics for non-spaced or non-Latin writing systems (e.g., Japanese Kana/Kanji).

- **Vocabulary Retention Handoff:** Export selected target words, translations, and context sentences directly into a designated Anki flashcard deck.

2 Stakeholders

2.1 Client

2.2 Customer

The primary customers are self-directed early-intermediate to intermediate language learners looking for an engaging method to acquire authentic vocabulary, as well as song-map content creators seeking an efficient authoring tool to build synchronized language lessons.

2.3 Other Stakeholders

Secondary stakeholders include:

- The LingoLyrics capstone development team (Omar Bakr, Raymond Ma, Rhea Gokhale, Aansh Kotian).
- Anki ecosystem users and open-source software maintainers.
- Copyright holders and public domain music providers.

2.4 Hands-On Users of the Project

- **Learners:** Individuals using the mobile app on iOS or Android devices with headphones to study music lessons.
- **Creators:** Language educators or advanced learners using the web editor on desktop/laptop web browsers to time-align lyrics and add annotations.

2.5 Personas

- **Kenji (Intermediate Learner):** An early-intermediate Japanese learner who loves Japanese pop music but struggles to read Kanji at singing

speed. He uses the mobile app to follow synchronized romanized text and export tricky vocabulary straight to his Anki deck.

- **Elena (Content Creator):** A language tutor who wants to create engaging song lessons for her students. She uses the web editor on her desktop to upload audio files, synchronize lyric timestamps, and attach contextual usage notes.

2.6 Priorities Assigned to Users

Primary development priority is given to the mobile language learner to ensure low-latency audio playback, responsive UI interaction, and reliable flashcard exports. Second priority is assigned to the web song-map creator interface.

2.7 User Participation

Representative learners and creators will participate in user testing sessions during the Winter term custom document deliverable (Usability Report) to evaluate workflow completion and interface learnability.

2.8 Maintenance Users and Service Technicians

The capstone development team acts as the primary maintainers. Continuous Integration (CI) test suites and JSON Schema validation ensure long-term maintainability and schema stability.

3 Mandated Constraints

3.1 Solution Constraints

- **Implementation Languages:** All application code and shared schemas must be written in TypeScript.
- **Frameworks:** Learner mobile application must be built using React Native with Expo. Creator web editor must be built using React.
- **Data Format:** Lessons must be stored and exchanged using a standardized, versioned JSON Schema song-map format.

3.2 Implementation Environment of the Current System

The mobile learner app targets mobile web and native platforms (iOS and Android) operated on smartphones with audio headphone output. The creator web editor runs on desktop browsers (Chrome, Firefox, Safari, Edge).

3.3 Partner or Collaborative Applications

- **Anki / AnkiDroid / AnkiConnect:** Third-party flashcard ecosystem targeted for vocabulary export payloads.
- **kuromoji.js:** JavaScript NLP tokenization library used for client-side Japanese segmentation and romanization.

3.4 Off-the-Shelf Software

- **Expo Audio / Howler.js:** Open-source audio engine libraries for mobile and web environments.
- **Jest & ESLint:** Testing framework and code quality tools.

3.5 Anticipated Workplace Environment

Learners will use the application in personal, quiet, or transit environments via mobile phones and headphones. Creators will author lessons in typical home or office environments on personal computers.

3.6 Schedule Constraints

Development must strictly align with McMaster Capstone milestones:

- SRS Document & Hazard Analysis: October 19, 2026
- Proof of Concept Demonstration: Late November 2026
- Revision 0 Demonstration: February 2027
- Final Documentation & EXPO: March 2027

3.7 Budget Constraints

The project operates under a zero-budget open-source model using standard capstone hardware and open-source tooling.

3.8 Enterprise Constraints

All code and documentation must be licensed under the MIT License and hosted publicly on GitHub.

4 Naming Conventions and Terminology

4.1 Glossary of All Terms, Including Acronyms, Used by Stakeholders involved in the Project

- **Anki:** A spaced-repetition flashcard application used for vocabulary review.
- **Contextual Annotation:** An explanation of a word, phrase, or lyric line capturing its intended meaning in a song.
- **JSON Schema:** A declarative language for validating the structure and content of JSON song-map data.
- **PoC (Proof of Concept):** A narrow prototype built to validate key technical risks prior to full development.
- **Segmentation:** The process of splitting unspaced text (e.g., Japanese) into discrete tokens or words.
- **Song Map:** A structured JSON file containing timed lyrics, translations, transliterations, annotations, and metadata connected to an audio track.
- **Transliteration:** Converting text from one script to another (e.g., Japanese Kana to Latin Romaji) to assist pronunciation.

5 Relevant Facts And Assumptions

5.1 Relevant Facts

- Music-based language learning increases student engagement and aids phrase retention.
- Languages like Japanese lack spaces between words, necessitating specialized NLP tokenization for selection interaction.
- Anki remains one of the most widely adopted open-source spaced-repetition platforms among serious language learners.

5.2 Business Rules

- **BR1 (Open Access):** No user accounts or paid subscriptions are required to access open-source song maps.
- **BR2 (Schema Adherence):** Any song map created by the web editor must pass automated JSON Schema validation before export or deployment.
- **BR3 (Copyright Compliance):** The application shall not package or distribute copyrighted commercial audio files without explicit licensing or public-domain status.

5.3 Assumptions

- Learners have access to modern smartphones (iOS or Android) and headphones.
- Creators have access to audio files and text lyrics for the songs they choose to map.
- Users intending to export cards have a local installation of Anki or AnkiDroid available.

6 The Scope of the Work

6.1 The Current Situation

Currently, language learners attempting song study must manually synchronize music playback on standard apps (e.g., Spotify) with external lyric translations and online dictionaries. This fragmented manual workflow inhibits active listening and breaks focus.

6.2 The Context of the Work

LingoLyrics bridges content creators and language learners through a unified song-map representation. Creators build structured maps using the web editor, which are then parsed and rendered dynamically by the mobile learner application during audio playback.

6.3 Work Partitioning

- **WP1 (Mobile Learner App):** Real-time audio playback, timed lyric synchronization, interactive token selection, translation display, and Anki export.
- **WP2 (Web Song-Map Editor):** Audio timeline alignment, lyric text editing, segmentation/transliteration generation, annotation attachment, and JSON Schema export.
- **WP3 (Shared Core):** Schema definitions, song-map validation logic, and shared TypeScript type definitions.

6.4 Specifying a Business Use Case (BUC)

- **BUC-1 (Authoring Song Map):** Creator uploads audio and lyrics $\rightarrow$ Editor assists in timing line boundaries $\rightarrow$ Creator adds annotations $\rightarrow$ System outputs valid JSON song map.
- **BUC-2 (Interactive Lesson Study):** Learner selects song map $\rightarrow$ Mobile app loads audio and map $\rightarrow$ Real-time line highlighting displays as audio plays $\rightarrow$ Learner taps word to view definition/annotation $\rightarrow$ Learner exports word to Anki.

7 Business Data Model and Data Dictionary

7.1 Business Data Model

The core domain model centers on the **Song Map** entity, which encapsulates:

- **Song Metadata:** Title, artist, target language, audio source URL.
- **Lyric Lines:** Start timestamp, end timestamp, original text, line translation.
- **Tokens / Words:** Segmented word, reading/transliteration, dictionary definition.
- **Annotations:** Target word/phrase range, contextual explanation, grammatical notes.

7.2 Data Dictionary

- **SongMapJSON:** Root object adhering to `songmap.schema.json`.
- **LyricLine:** Object containing `lineId`, `startTime`, `endTime`, `rawText`, and `translatedText`.
- **VocabularyPayload:** Export object sent to Anki containing `term`, `translation`, and `contextSentence`.

8 The Scope of the Product

8.1 Product Boundary

The product boundary includes the React Native Mobile Learner Application and the React Web Song-Map Editor. External systems outside the boundary include local Anki desktop/AnkiDroid instances and third-party remote audio storage.

8.2 Product Use Case Table

PUC ID	Name	Primary Actor	Outcome
PUC-1	Play Synchronized Lesson	Learner	Real-time lyric tracking within 0.5s tolerance.
PUC-2	Inspect Word Annotation	Learner	Displays translation, reading, and contextual note.
PUC-3	Export Term to Anki	Learner	Term, definition, and context sentence added to deck.
PUC-4	Author Song Map	Creator	Valid JSON Song Map generated and downloaded.

8.3 Individual Product Use Cases (PUC's)

- **PUC-1 (Play Synchronized Lesson):** The learner selects a song map; the application streams audio and continuously updates highlighted lyric lines in sync with playback time.
- **PUC-3 (Export Term to Anki):** Learner taps "Add to Anki" on a selected token; app formats a `VocabularyPayload` and transmits it via API/payload to the local Anki instance.

9 Functional Requirements

9.1 Functional Requirements

- **FR1 (Audio Playback):** The mobile app shall play audio tracks while reporting sub-second playback time positions continuously.

- **FR2 (Synchronized Highlighting):** The app shall visually highlight current lyric lines within 0.5 seconds of the mapped timestamp.
- **FR3 (Text Segmentation):** The app/editor shall segment non-spaced scripts (e.g., Japanese) into selectable discrete word units.
- **FR4 (Transliteration Display):** The app shall display transliterations (e.g., Romaji/Furigana) above or beside target words.
- **FR5 (Contextual Notes):** The app shall display attached cultural or grammatical annotations when an annotated target is tapped.
- **FR6 (Anki Export):** The system shall export selected vocabulary with its target word, translation, and context line into an Anki deck.
- **FR7 (Schema Validation):** The web editor shall validate exported song maps against the shared JSON Schema before download.

10 Look and Feel Requirements

10.1 Appearance Requirements

Insert your content here.

10.2 Style Requirements

Insert your content here.

11 Usability and Humanity Requirements

11.1 Ease of Use Requirements

Insert your content here.

11.2 Personalization and Internationalization Requirements

Insert your content here.

11.3 Learning Requirements

Insert your content here.

11.4 Understandability and Politeness Requirements

Insert your content here.

11.5 Accessibility Requirements

Insert your content here.

12 Performance Requirements

12.1 Speed and Latency Requirements

Insert your content here.

12.2 Safety-Critical Requirements

Insert your content here.

12.3 Precision or Accuracy Requirements

Insert your content here.

12.4 Robustness or Fault-Tolerance Requirements

Insert your content here.

12.5 Capacity Requirements

Insert your content here.

12.6 Scalability or Extensibility Requirements

Insert your content here.

12.7 Longevity Requirements

Insert your content here.

13 Operational and Environmental Requirements

13.1 Expected Physical Environment

Insert your content here.

13.2 Wider Environment Requirements

Insert your content here.

13.3 Requirements for Interfacing with Adjacent Systems

Insert your content here.

13.4 Productization Requirements

Insert your content here.

13.5 Release Requirements

Insert your content here.

14 Maintainability and Support Requirements

14.1 Maintenance Requirements

Insert your content here.

14.2 Supportability Requirements

Insert your content here.

14.3 Adaptability Requirements

Insert your content here.

15 Security Requirements

15.1 Access Requirements

Insert your content here.

15.2 Integrity Requirements

Insert your content here.

15.3 Privacy Requirements

Insert your content here.

15.4 Audit Requirements

Insert your content here.

15.5 Immunity Requirements

Insert your content here.

16 Cultural Requirements

16.1 Cultural Requirements

Insert your content here.

17 Compliance Requirements

17.1 Legal Requirements

Insert your content here.

17.2 Standards Compliance Requirements

Insert your content here.

18 Open Issues

Insert your content here.

19 Off-the-Shelf Solutions

19.1 Ready-Made Products

Insert your content here.

19.2 Reusable Components

Insert your content here.

19.3 Products That Can Be Copied

Insert your content here.

20 New Problems

20.1 Effects on the Current Environment

Insert your content here.

20.2 Effects on the Installed Systems

Insert your content here.

20.3 Potential User Problems

Insert your content here.

20.4 Limitations in the Anticipated Implementation Environment That May Inhibit the New Product

Insert your content here.

20.5 Follow-Up Problems

Insert your content here.

21 Tasks

21.1 Project Planning

Development tasks are partitioned among team members according to component ownership:

- Learner App Sync & Playback: Omar Bakr
- Song-Map Editor & Schema: Raymond Ma
- Language Processing & Segmentation: Rhea Gokhale
- Anki Integration & CI/CD Pipeline: Aansh Kotian

21.2 Planning of the Development Phases

Project deliverables are managed through GitHub Projects Kanban boards following a phased release schedule:

- **Phase 1 (Fall - PoC):** Core audio sync, Japanese segmentation, and Anki handoff validation.
- **Phase 2 (Winter - Rev 0):** Complete mobile application UI, full web editor features, and usability testing.
- **Phase 3 (Spring - Rev 1):** Final integration polish, documentation package, and EXPO demonstration.

22 Migration to the New Product

22.1 Requirements for Migration to the New Product

LingoLyrics is a new product and does not replace an existing deployed application. Therefore, no migration of user accounts, lesson libraries, or legacy application data is required for the initial release.

The initial release shall allow creators to produce new song maps through the web editor and allow learners to open song maps that conform to the shared, versioned JSON Schema. Future versions may introduce song-map import or conversion tooling if an earlier schema version or an external lesson format must be supported.

22.2 Data That Has to be Modified or Translated for the New System

No existing production data must be converted for the initial release. Each initial lesson will be authored directly in the shared song-map format and validated before use by the learner application.

If song-map schema revisions are introduced in a future release, a compatible reader or documented conversion process will be required so that previously created valid song maps remain usable. Translation, transliteration, timing, and contextual annotation data are stored as explicit song-map fields and do not require conversion during the initial release.

23 Costs

Insert your content here.

24 User Documentation and Training

24.1 User Documentation Requirements

Insert your content here.

24.2 Training Requirements

Insert your content here.

25 Waiting Room

Insert your content here.

26 Ideas for Solution

The proposed solution consists of a mobile learner application, a web-based song-map editor, and a shared TypeScript core package. The learner application will play lesson audio, display synchronized lyric lines, provide translations, transliterations, and contextual annotations, and prepare selected vocabulary for export to Anki. The editor will support creating and validating song maps without requiring creators to manually edit JSON files.

A versioned JSON Schema will define the shared song-map representation, including song metadata, timed lyric lines, selectable tokens, translations, transliterations, and annotations. Using one validated format allows lessons created in the editor to be opened by the learner application without manual changes.

The current implementation direction is React Native with Expo for the learner application, React for the creator editor, and TypeScript for the shared schema and validation logic. Japanese segmentation and romanization may use a client-side language-processing library such as `kuromoji.js`. These are implementation ideas rather than additional product requirements; equivalent technologies may be used provided the stated requirements and constraints are satisfied.

27 Appendix — Generative AI Use Disclosure

27.1 AI Tools Used

OpenAI Codex, using the GPT-5.6 Terra model, was utilized through the Codex desktop application.

27.2 How and Where Used

Codex was used to assist in drafting, structuring, and verifying Volere SRS section coverage based on primary project context files from the repository. Specifically, it aided in formulating functional requirements, specifying business use cases, and structuring data model dictionaries.

27.3 Verification of AI-Generated Content

All AI-assisted content was strictly checked and verified by team members against project constraints and supervisor guidelines:

- Verified that all requirement fit criteria are measurable and align with the core MVP goals.
- Ensured all technical tool choices (React Native, TypeScript, kuromoji.js) remain consistent across all repository documentation.

28 Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?